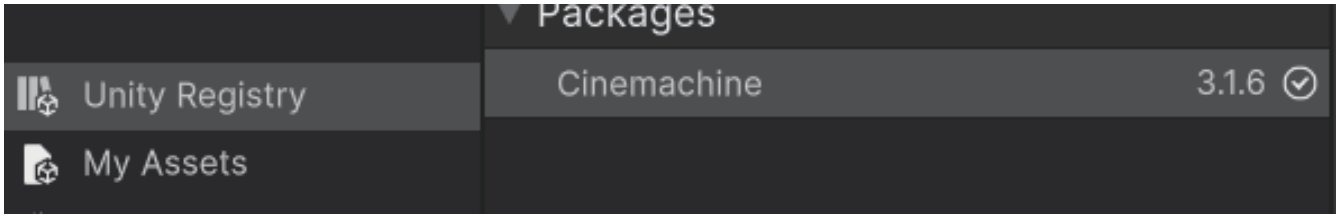
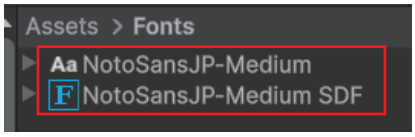
ProjectSetting から「CinemaChine」の導入

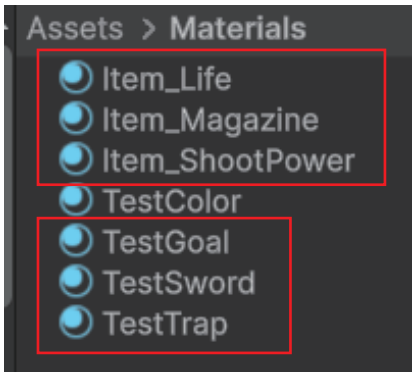


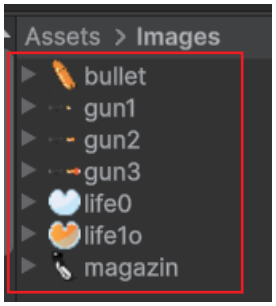
必要なスクリプトのインポート

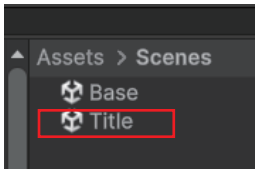
・CinemaChineController.cs　　リザルト時・リトライ時のカメラワーク切り替え
・GameManager.cs　ゲームステータスを定義、シーン切り替えのメソッドを容易
・Item.cs　アイテムのスクリプト（３種類 マガジン、銃のパワーアップ、体力回復）
・NormalSword.cs　ソードの実装 ※ソード発生中はジャンプもショットもできない
・ObjectGenerator.cs　エネミー・トラップ・アイテム・ゴールの生成
・ScoreManager.cs　ステージスコアとトータルスコアの計算
・UIController.cs　各 UI を更新するためのメソッドなど
・TitleManager.cs　タイトル画面のスクリプト

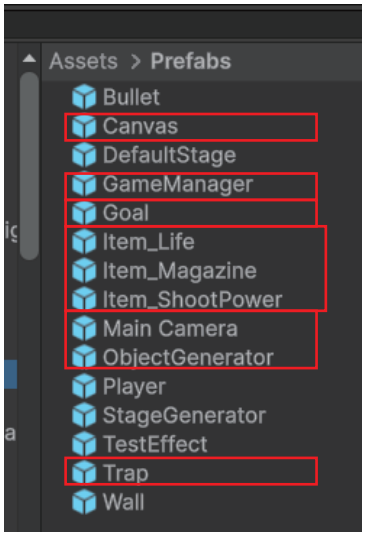
必要なアセットのインポート

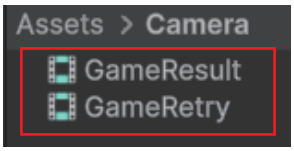


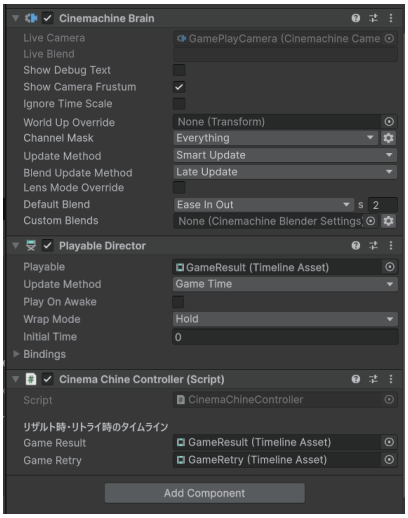






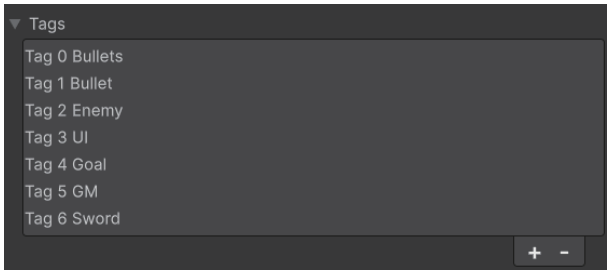


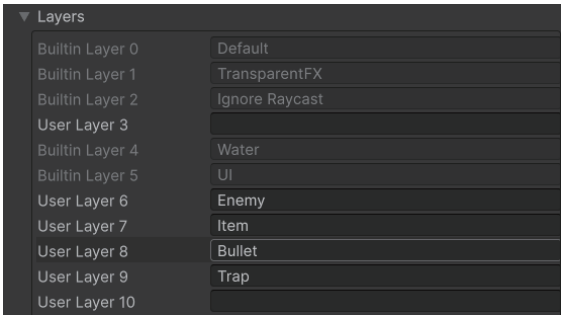


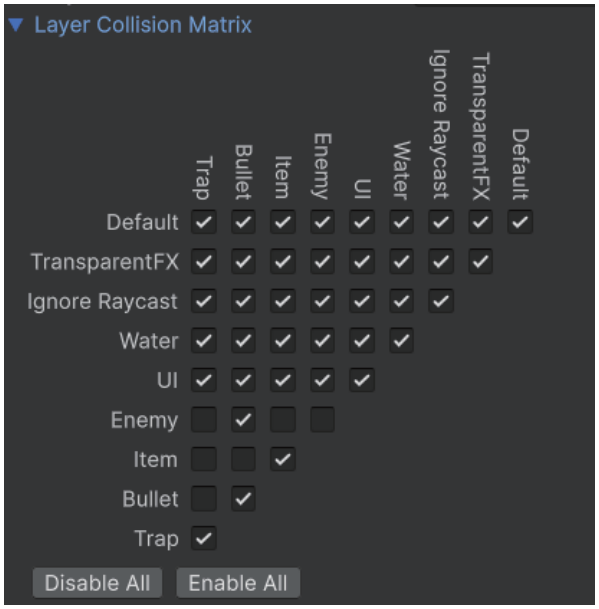


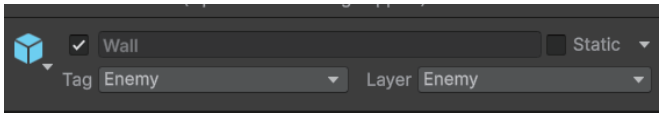
MainCamera はプレハブと差し替え予定

タグとレイヤーの調整

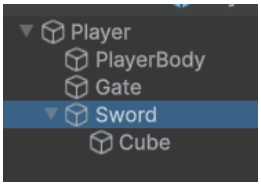


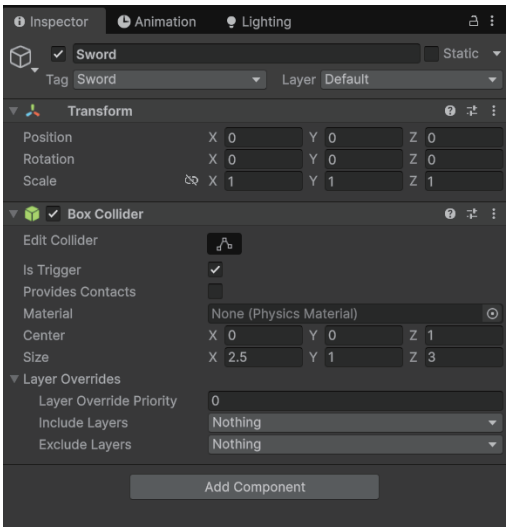






Player オブジェクトの調整「Sword オブジェクトと NormalSword.cs のアタッチ」





PlayerRun の更新

```
[Header("ソードのスクリプト")]
public NormalSword normalSword; //ソード中の動きを封じるため
```

```
//体力の回復
public void LifeUP()
{
    life++;
    if (life > DefaultLife) life = DefaultLife;
    GameObject canvas = GameObject.FindGameObjectWithTag("UI"); //UIタグの検索
    canvas.GetComponent<UIController>().UpdateLife(Life()); //UIの更新
}
```

```
//体力のダメージによる減少
public void LifeDown()
{
    life--;
    GameObject canvas = GameObject.FindGameObjectWithTag("UI"); //UIタグの検索
    canvas.GetComponent<UIController>().UpdateLife(Life()); //UIの更新
}
```

```
void OnMove(InputValue value)
{
    if (normalSword.GetIsSword()) return; //ソード中なら何もできない

    if (resetIntervalCol == null)
    {
        Vector2 inputVector = value.Get<Vector2>();
        currentMoveInputX = inputVector.x; // x成分が左右の入力（-1:左, 0:なし, 1:右）
    }
}
```

```
void OnJump(InputValue value)
{
    if (normalSword.GetIsSword()) return; //ソード中なら何もできない

    Jump();
}
```

```
void Update()
{
    if (GameManager.gameState == GameState.stageclear || GameManager.gameState == GameState.result) return;
```

```
void OnControllerColliderHit(ControllerColliderHit hit)
{
    if (IsStun()) return;

    if (hit.gameObject.tag == "Enemy")
    {
        LifeDown(); //体力減少
        GetComponent<NormalShooter>().ShootPowerDown(); //威力も減らす
        recoverTime = StunDuration; //硬直カウントダウンの開始

        //もし体力0ならゲームオーバー
        if (life <= 0) GameManager.gameState = GameState.gameover;

        //animator.SetTrigger("damage");

        //相手のエフェクト発生（と消滅）を発動
        hit.gameObject.GetComponent<Wall>().CreateEffect();
    }
}
```

```
//ゴールに触れたらステータスをゲームクリアに変更
void OnTriggerEnter(Collider other)
{
    if(other.gameObject.tag == "Goal")
    {
        GameManager.gameState = GameState.stageclear;
    }
}
```

追加された変数など微調整

※スクリプト更新後

Wall に Sword によるダメージを追加

```
[Header("スコア点数")]
public int point = 100;
```

```
//衝突
void OnTriggerEnter(Collider other)
{
    if (currentDamage != null) return; //ダメージコルーチン中ならキャンセル

    //衝突相手が「Bullet」の場合
    if(other.gameObject.tag == "Bullet" || other.gameObject.tag == "Sword")
    {
        string tag = other.gameObject.tag;
        //ダメージコルーチンを発動
        currentDamage = StartCoroutine(DamageCol(tag));
        if (life <= 0)  //lifeが残っていなければ消滅
        {
            ScoreManager.ScoreUp(point); //撃破によるスコアアップ
            CreateEffect(); //エフェクト処理と削除
        }
    }
}
```

```
//ダメージコルーチン
IEnumerator DamageCol(string tag)
{
    GameObject player = GameObject.FindGameObjectWithTag("Player");

    //耐久力を減らす
    if (tag == "Bullet")
    {
        life -= player.gameObject.GetComponent<NormalShooter>().GetShootPower();
    }
    else if(tag == "Sword")
    {
        life -= player.gameObject.GetComponent<NormalSword>().GetSwordPower();
    }
    yield return new WaitForSeconds(damegeTime);
    currentDamage = null; //コルーチン参照を解放
    yield return new WaitForSeconds(0.1f);
    //振動していたボディをもとの位置に戻す
    damageBody.transform.localPosition = new Vector3(0, 0, 0);
}
```

BulletManager の更新

```
[Header("UIオブジェクト")]
public UIController ui;
```

```
//弾の消費
public void ConsumeBullet()
{
    if (bulletRemaining > 0) //残弾があれば
    {
        bulletRemaining--; //弾を消費
        ui.UpdateBullet(); //UIを更新
    }
}
```

```
//マガジン数の取得
public int GetMagazineRemaining()
{
    return magazine; //現状の残弾を返す
}
```

```
//弾の充填
public void AddBullet(int num)
{
    bulletRemaining = maxRemaining;
    ui.UpdateBullet(); //UIを更新
}

//マガジンの補充
public void AddMagazine()
{
    magazine++;
    ui.UpdateMagazine();    //UIを更新
}
```

```
//充填メソッド
public void RecoverBullet()
{
    //充填コルーチンがすでに走っていれば何もしない
    if (bulletRecover == null)
    {
        if (magazine > 0) //マガジンの残数があれば補充可能
        {
            magazine--; //マガジンは消費
            ui.UpdateMagazine(); //UI更新

            //補充コルーチンの発動(Coroutine型の変数に発動したコルーチ
            //※Coroutine型の変数が何かを参照していれば、すでにコルーチ
              の終わりに解放予定)
            bulletRecover = StartCoroutine(RecoverBulletCol());
        }
    }
}
```

```
//充填コルーチン
IEnumerator RecoverBulletCol()
{
    ui.Reloding(); //UI（リロード中）を発動

    //カウンターの数字を整える
    counter = recoveryTime;

    while(counter > 0)
    {
        yield return new WaitForSeconds(1.0f); //ウェイト処理
        counter--;
    }
    AddBullet(maxRemaining);//弾数充填

    //充填が終わったのでCoroutine型の変数を解放
    //※またコルーチンが発動できるようにする
    bulletRecover = null;
}
```

＋void OnGUI() の無効化

NormalShooter の更新

```
const int maxShootPower = 3;  //最大威力
int shootPower = 1;  //現在威力
```

```
[Header("ソードのスクリプト")]
public NormalSword normalSword; //ソード中の動きを封じるため
```

```
//InputAction(Playerマップ)のAttackアクションがおされたら
void OnAttack(InputValue value)
{
    if (normalSword.GetIsSword()) return;  //ソード中なら何もできない

    //リトライ状態の時ならやり直し
    if (GameManager.gameState == GameState.retry)
    {
        GameManager.RetryScene();
    }
    else if(GameManager.gameState == GameState.result) //リザルト状態の時ならネクスト
    {
        GameManager gm = GameObject.FindGameObjectWithTag("GM").GetComponent<GameManager>();
        gm.NextScene(gm.nextScene);
    }
    else //ゲームステータスがプレイ中なら
    {
        Shoot();
    }
}
```

```
//威力を上げる
public void ShootPowerUp()
{
    shootPower++; //威力を上げる
    if (shootPower > maxShootPower) shootPower = maxShootPower; //最大威力までに抑える
    GameObject canvas = GameObject.FindGameObjectWithTag("UI"); //UIタグの検索
    canvas.GetComponent<UIController>().UpdateGun(); //UIの更新
}

//威力が下がる
public void ShootPowerDown()
{
    shootPower--;//威力を下がる
    if (shootPower <= 0) shootPower = 1;//最小でも1
    GameObject canvas = GameObject.FindGameObjectWithTag("UI");//UIタグの検索
    canvas.GetComponent<UIController>().UpdateGun();//UIの更新
}

//現在威力の取得
public int GetShootPower()
{
    return shootPower;
}
```